ПРАКТИКА 11

ВЫПОЛНИЛА СТУДЕНТ 1-МД-35

БАБАЯН ЭЛЛЕН

1. ТОП 5 IP-АДРЕСОВ ПО ЧАСТОТЕ ЗАПРОСОВ

```python
#Выполнила студент 1-мд-35 Бабаян Эллен
from collections import Counter
import re

def get_top_ips(filename):  1 usage
    ''' Функция для подсчёта топ-5 IP-адресов '''
    ip_counter = Counter()
    total_request = 0
    with open(filename, 'r', encoding='utf-8') as file:
        for line in file:
            if line.strip():
                ip = line.split()[0]
                ip_counter[ip] += 1
                total_request += 1
    print("Топ-5 IP- адресов:")
    print("-" * 40)
    for ip, count in ip_counter.most_common(5):
        percentage = (count / total_request) * 100
```

```python
def get_top_ips(filename):  1 usage
    with open(filename, 'r', encoding='utf-8') as file:
                ip_counter[ip] += 1
                total_request += 1
    print("Топ-5 IP- адресов:")
    print("-" * 40)
    for ip, count in ip_counter.most_common(5):
        percentage = (count / total_request) * 100
        print(f"{ip} - {count} запросов ({percentage:.1f}%)")


get_top_ips('logs.txt')
```

```
pflfybt 1 ×
C:\Users\PC\PyCharmMiscProject\.venv\Scripts\python
Топ-5 IP- адресов:
----------------------------------------
127.0.0.1 - 15 запросов (29.4%)
10.0.0.5 - 14 запросов (27.5%)
192.168.1.10 - 9 запросов (17.6%)
172.16.0.1 - 7 запросов (13.7%)
192.168.1.20 - 6 запросов (11.8%)

Process finished with exit code 0
```

Counter() - специальный словарь для подсчета количества элементов
line.split()[0] - берем первое слово в строке (это IP-адрес)
most_common(5) - получаем 5 самых частых элементов
Процент считаем: (количество_ip / всего_запросов) * 100

2. ГРУППИРОВКА ПО СТАТУС-КОДАМ

```python
#2
from collections import Counter

def get_status_codes(filename):  1 usage
    """
    Функция для подсчета запросов по статус-кодам
    """
    status_counter = Counter()
    total_requests = 0

    with open(filename, 'r', encoding='utf-8') as file:
        for line in file:
            if line.strip():
                # Разбиваем строку на части
                parts = line.split()

                status = parts[-2]
                status_counter[status] += 1
                total_requests += 1
```

```python
def get_status_codes(filename):  1 usage
    with open(filename, 'r', encoding='utf-8') as file:
                total_requests += 1

    print("Статус-коды:")
    print("-" * 40)

    for status in sorted(status_counter.keys()):
        count = status_counter[status]
        percentage = (count / total_requests) * 100
        print(f"{status} - {count} запросов ({percentage:.1f}%)")


# Использование
get_status_codes('logs.txt')
```

```
Статус-коды:
----------------------------------------
0 - 5 запросов (9.8%)
11234 - 1 запросов (2.0%)
123 - 3 запросов (5.9%)
1234 - 3 запросов (5.9%)
12345 - 1 запросов (2.0%)
1542 - 1 запросов (2.0%)
1567 - 1 запросов (2.0%)
1890 - 1 запросов (2.0%)
234 - 2 запросов (3.9%)
2345 - 3 запросов (5.9%)
23456 - 1 запросов (2.0%)
2456 - 1 запросов (2.0%)
3210 - 1 запросов (2.0%)
345 - 2 запросов (3.9%)
3456 - 2 запросов (3.9%)
456 - 3 запросов (5.9%)
4567 - 1 запросов (2.0%)
```

```
4567 - 1 запросов (2.0%)
512 - 1 запросов (2.0%)
54321 - 1 запросов (2.0%)
543210 - 1 запросов (2.0%)
567 - 3 запросов (5.9%)
5678 - 1 запросов (2.0%)
67 - 1 запросов (2.0%)
678 - 1 запросов (2.0%)
6789 - 1 запросов (2.0%)
789 - 3 запросов (5.9%)
7890 - 1 запросов (2.0%)
876 - 1 запросов (2.0%)
890 - 1 запросов (2.0%)
8901 - 1 запросов (2.0%)
892 - 1 запросов (2.0%)
98765 - 1 запросов (2.0%)

Process finished with exit code 0
```

Что такое коды ответов (статус-коды)?

Статус-коды - это трехзначные числа, которые сервер возвращает в ответ на запрос. Они показывают, успешно ли обработан запрос или произошла ошибка.

Основные группы статус-кодов:

2xx (200-299) - Успех (запрос обработан)

3xx (300-399) – Перенаправление

4xx (400-499) - Ошибка клиента (например, страница не найдена)

5xx (500-599) - Ошибка сервера

```
127.0.0.1 -- [24/Feb/2026:11:17:00 +0300] "GET /api/users HTTP/1.1" 200 1542 "-"
                                                                        ↑
                                                                Статус-код (200)
```

3. СОЗДАТЬ ТЕСТОВЫЕ ДАННЫЕ ПРЯМО В ПРОГРАММЕ

```python
# 3
from collections import Counter
import re


def get_top_paths(filename):  1 usage
    """
    Функция для поиска самых популярных путей запросов
    """
    path_counter = Counter()

    path_pattern = re.compile(r'"[A-Z]+ ([^ ]+) HTTP')

    with open(filename, 'r', encoding='utf-8') as file:
        for line in file:
            if line.strip():
                match = path_pattern.search(line)
                if match:
                    path = match.group(1)  # group(1) - это то, что в скобках
                    path_counter[path] += 1

    print(" Самые популярные пути:")
    print("-" * 40)

    for path, count in path_counter.most_common(3):
        print(f"{path} - {count} запросов")


# Использование
get_top_paths('logs.txt')
```

```
 Самые популярные пути:
----------------------------------------
/api/orders - 2 запросов
/api/users - 1 запросов
/login - 1 запросов

Process finished with exit code 0
```

- Регулярное выражение: r'"[A-Z]+ ([^]+) HTTP'
  - "[A-Z]+ - кавычка и большие буквы (GET, POST и т.д.)
  - ([^]+) - пробел, затем всё кроме пробела (это путь), затем пробел
  - HTTP - слово HTTP
- match.group(1) - извлекаем только путь (то, что в скобках)

4. ОБЩИЙ РАЗМЕР ОТВЕТОВ

```python
#4
def get_total_size_fixed(filename):  1 usage
    """
    Размер находится на позиции -2!
    """
    total_size = 0
    count = 0
    line_number = 0

    print("🔍 Анализ размеров ответов:")
    print("-" * 50)

    with open(filename, 'r', encoding='utf-8') as file:
        for line in file:
            line_number += 1
            line = line.strip()
            if not line:
                continue

            parts = line.split()

            size_str = parts[-2]

            size_str = size_str.strip('"')

            try:
                size = int(size_str)
                total_size += size
                count += 1
                print(f"✓ Строка {line_number:2d}: размер = {size:6d} байт")
            except ValueError:
                print(
```

```python
                    f"× Строка {line_number:2d}: НЕ ЧИСЛО! '{size_str}' (индекс {parts.index(size_str) if size_str in parts else '?'})")
                print(f"  Все части: {parts}")
                break

    print("\n" + "=" * 50)
    print(f" Найдено размеров: {count}")
    print(f" Общий размер: {total_size:,} байт")

    if total_size > 0:
        if total_size > 1024 * 1024:
            print(f" Это: {total_size / (1024 * 1024):.2f} MB")
        elif total_size > 1024:
            print(f" Это: {total_size / 1024:.2f} KB")

    return total_size


# Запуск
get_total_size_fixed('logs.txt')
```

```
🔍 Анализ размеров ответов:
--------------------------------------------------
✓ Строка  1: размер =   1542 байт
✓ Строка  2: размер =    892 байт
✓ Строка  3: размер =   2456 байт
✓ Строка  4: размер =   3210 байт
✓ Строка  5: размер =    456 байт
✓ Строка  6: размер =    789 байт
✓ Строка  7: размер =    234 байт
✓ Строка  8: размер =      0 байт
✓ Строка  9: размер =   5678 байт
✓ Строка 10: размер =   2345 байт
✓ Строка 11: размер =    123 байт
✓ Строка 12: размер =    876 байт
✓ Строка 13: размер =   1890 байт
✓ Строка 14: размер =    512 байт
✓ Строка 15: размер =    345 байт
✓ Строка 16: размер =  11234 байт
✓ Строка 17: размер =     67 байт
✓ Строка 18: размер =    456 байт
✓ Строка 19: размер =    789 байт
✓ Строка 20: размер =    234 байт
✓ Строка 21: размер =  54321 байт
✓ Строка 22: размер =      0 байт
✓ Строка 23: размер =   1234 байт
✓ Строка 24: размер =    678 байт
✓ Строка 25: размер =   1567 байт
✓ Строка 26: размер =   1234 байт
✓ Строка 27: размер =   2345 байт
✓ Строка 28: размер =    123 байт
✓ Строка 29: размер =   3456 байт
✓ Строка 30: размер =      0 байт
✓ Строка 31: размер =  98765 байт
✓ Строка 32: размер =   2345 байт
✓ Строка 33: размер =    567 байт
✓ Строка 34: размер =   7890 байт
✓ Строка 35: размер =    456 байт
✓ Строка 36: размер =   6789 байт
✓ Строка 37: размер =  12345 байт
✓ Строка 38: размер =      0 байт
✓ Строка 39: размер =  23456 байт
✓ Строка 40: размер =    890 байт
✓ Строка 41: размер =    345 байт
✓ Строка 42: размер =    567 байт
✓ Строка 43: размер =    789 байт
✓ Строка 44: размер =    123 байт
✓ Строка 45: размер = 543210 байт
✓ Строка 46: размер =   1234 байт
✓ Строка 47: размер =    567 байт
✓ Строка 48: размер =   4567 байт
✓ Строка 49: размер =      0 байт
✓ Строка 50: размер =   3456 байт
✓ Строка 51: размер =   8901 байт

==================================================
 Найдено размеров: 51
 Общий размер: 816,348 байт
 Это: 797.21 KB

Process finished with exit code 0
```

```
127.0.0.1 -- [24/Feb/2026:11:17:00 +0300] "GET /api/users HTTP/1.1" 200 1542 "-"
                                                                      ^^^^
                                                                Это РАЗМЕР (154
2)
```

1. Регулярные выражения

```
# Что такое регулярные выражения?
pattern = r'"[A-Z]+ ([^ ]+) HTTP"'
# r'' - сырая строка (не интерпретирует \ как
        экранирование)
# [A-Z]+ - одна или несколько заглавных букв
# [^ ]+ - один или несколько символов, кроме пробела
# () - группа для извлечения
```

2. Counter из collections

```
from collections import Counter

# Counter автоматически подсчитывает количество
        элементов
counter = Counter()
counter['apple'] += 1  # теперь {'apple': 1}
counter['apple'] += 1  # теперь {'apple': 2}
counter['banana'] += 1  # теперь {'apple': 2, 'banana': 1}

# most_common() сортирует по убыванию
print(counter.most_common(1))  # [('apple', 2)]
```

3. Разбиение строк (split)

```
line = '127.0.0.1 -- [date] "GET /path" 200 1542'
parts = line.split()  # Разбиваем по пробелам
# parts = ['127.0.0.1', '--', '[date]', '"GET', '/path"', '200',
'1542']
```

```
# Индексация с конца
last = parts[-1]      # последний элемент: '1542'
second_last = parts[-2]  # предпоследний: '200'
```

4. Форматирование строк

```
# f-строки (Python 3.6+)
name = "IP"
count = 42
percent = 50.5
print(f"{name} - {count} ({percent:.1f}%)")  # IP - 42 (50.5%)

# Форматирование с выравниванием
print(f"{name:<10} - {count:>5}")  # IP         -    42
# <10 - выравнивание влево на 10 символов
# >5 - выравнивание вправо на 5 символов
```